

MudranSetu

Project brief for Claude Code: LAN printer sharing across Windows 7 / 8 / 10 / 11

Working name: MudranSetu (Hindi: *mudran* = printing, *setu* = bridge). If the owner picks a different name, do a case-sensitive find-and-replace across the repo. **Owner:** the Sub Postmaster of an India Post sub office; a capable non-programmer who will test on real machines and paste results back. **Status:** approved by the Divisional IT side; the tool is for in-office use only.

0. How Claude Code should use this document

- Read the whole document once before writing any code.
- Create **CLAUDE.md** in the repo root by condensing sections 3, 4, 6 and 12 (a starter is in Appendix A). Keep it under 120 lines.
- Work strictly milestone by milestone (section 9). Finish, build, and hand over test steps for one milestone before touching the next.
- Claude Code cannot run Windows print APIs or see a printer. The owner runs each build on real PCs/VMs and pastes logs. Design every feature so that it can be verified from a log file or a Diagnostics button.
- When a hard constraint (section 3) seems to block progress, stop and ask the owner; do not silently switch framework, ports, or add dependencies.
- Prefer boring, well-known Windows APIs over clever ones. Simplicity beats features: the users are postal clerks, not IT staff.

1. Background and the problem

The office is a small government sub post office. All desktop PCs are on one Ethernet LAN behind a server/switch. The machines run a mix of **Windows 7 SP1**, **Windows 8/8.1**, **Windows 10** and **Windows 11**; some Windows 7 boxes are 32-bit. Printers are (a) older USB printers plugged into one PC and (b) newer Wi-Fi/network printers.

Built-in Windows printer sharing fails across this mix for reasons that no amount of clicking fixes:

- **Drivers per OS:** every client needs a driver built for its own Windows version. Old USB printers often have no Windows 11 driver, and new printers have no Windows 7 driver.
- **SMB mismatch:** Windows 7 shares over SMB1, which Windows 10/11 disable by default; enabling SMB1 is a security downgrade the IT side does not want.
- **PrintNightmare hardening (2021):** connecting to a shared printer now needs admin rights to install its driver, and credential prompts appear constantly on workgroup PCs.
- **HomeGroup is gone** from Windows 10 (1803+) and 11, and Windows 7 never had the newer sharing UI.

Goal in one sentence: any PC in the office can print to any printer in the office, whatever Windows version either side runs, with no driver installation and no Windows credentials on the client side.

2. Goals and non-goals

Goals (MVP)

- Share any printer attached to any office PC (USB, network, or Wi-Fi) with any other office PC.
- Zero driver installation on client PCs; zero username/password prompts.
- Works fully offline on the LAN. Never contacts the internet.
- One EXE, one installer, installable by a non-technical person in under five minutes.
- Runs on Windows 7 SP1 (32 and 64-bit) through Windows 11.
- Plain-language status messages ("Sent to Counter-1 PC, printing", "Host PC is switched off").

Non-goals (do not build unless asked)

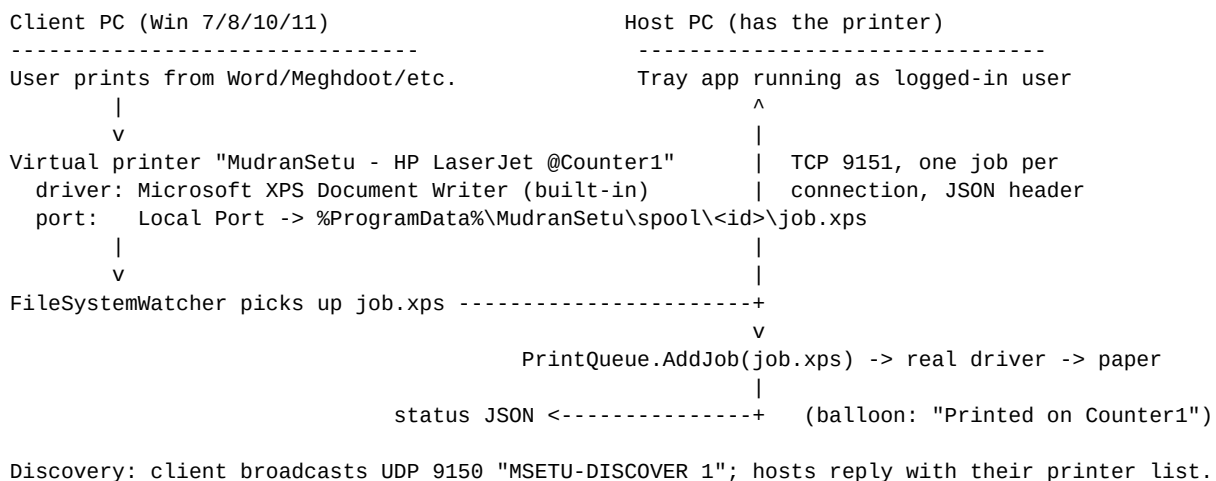
- Cloud, internet, mobile, macOS or Linux support.
- Scanning, print accounting, quotas, or a web dashboard.
- Replacing Windows sharing where it already works; both can coexist.
- Auto-update from the internet. Updates are done by running a new installer.

3. Hard constraints (never change without asking)

Item	Requirement	Why
Language / runtime	C#, .NET Framework 4.8 . Not .NET Core, not .NET 5-8+.	.NET 8 dropped Windows 7. Framework 4.8 is the only runtime that runs on all four OS versions.
UI	WinForms, single window with tabs, minimises to the system tray.	Ships inside Framework 4.8; no extra runtime.
CPU target	AnyCPU, "Prefer 32-bit" unchecked .	Some Windows 7 PCs are 32-bit; 64-bit PCs must still load 64-bit print DLLs.
Dependencies	Prefer none. Allowed: Newtonsoft.Json. Anything else: ask first. Every package must target net48 or netstandard2.0.	Fewer moving parts on old machines.
Network	LAN only. Bind to all local IPv4 interfaces. No outbound internet calls, no telemetry, no crash reporting to any server.	Government office policy.
Privileges	Admin (UAC) only during install and when creating/removing a virtual printer. Normal running: standard user.	Clerks do not have admin passwords.
Data locations	Config: %ProgramData%\MudranSetu\config.json. Spool: %ProgramData%\MudranSetu\spool\ Logs: %ProgramData%\MudranSetu\logs\ (daily files, keep 14 days).	Shared, non-roaming, survives user switch.
Strings	All user-visible text in Strings.resx from day one.	Hindi UI is a Phase 2 item.
Licence	MIT licence file in repo root.	Owner may share it with other post offices.

4. Architecture

One application, two roles. Any PC can be a **Host** (has a printer to share), a **Client** (wants to use a remote printer), or both.



Projects in the solution

- **MudranSetu.Core** (class library): protocol, discovery, spool watcher, printing, config, logging. No UI references.
- **MudranSetu.App** (WinForms exe): tray app with tabs *Share my printers*, *Use shared printers*, *Settings*, *Diagnostics*.
- **MudranSetu.Tests** (MSTest or xUnit, net48): unit tests for framing, discovery parsing, config, file-stability logic.
- **installer/MudranSetu.iss**: Inno Setup script (see section 10).

Why this design

- The XPS Document Writer exists on every Windows from 7 to 11, so the client never needs the real printer's driver.
- The host prints the XPS with its own, already-working driver. Windows converts XPS to GDI for old drivers automatically.
- No SMB, no shares, no Windows accounts. Only two ports, both owned by our app.
- Phase 2 adds a raw port-9100 relay for clients that *do* have a driver (better for dot-matrix and label printers).

5. Print pipeline, step by step

5.1 Client: creating the virtual printer (needs one-time UAC elevation)

- Add a **Local Port** whose name is a file path:
%ProgramData%\MudranSetu\spool\{printerId}\job.xps. Use the winspool XcvData API with the "Local Port" monitor and command "AddPort" (works on Win 7+). Fallback on Win 8+: PowerShell `Add-PrinterPort -Name <path>`.
- Call `AddPrinter` (winspool.drv) with driver name "**Microsoft XPS Document Writer v4**" on Windows 8+ and "**Microsoft XPS Document Writer**" on Windows 7. Detect which exists via `EnumPrinterDrivers`.
- Printer name pattern: `MudranSetu - {friendlyName} @{hostName}`. Set default paper to A4 in `DEVMODE`.
- Store `printerId`, `hostName`, `hostIp`, `port` and local printer name in `config.json` so the mapping survives restarts.
- Removing a remote printer must delete the printer, then the port, then the spool folder (in that order).

5.2 Client: catching the job

- Watch the spool folder with `FileSystemWatcher` plus a 2-second polling fallback (`FileSystemWatcher` is flaky on some Win 7 boxes).
- A job is ready when the file can be opened with exclusive share and its size has not changed for 1 second.
- Immediately rename it to `job-{guid}.xps` so the next print does not overwrite it, then send.
- Keep sent files for 1 hour, then delete. Keep the last 20 failed ones for diagnostics.
- If the host is unreachable: retry 3 times over 30 seconds, then show a balloon and keep the file in a *pending* folder for a manual "Retry" button.

5.3 Host: printing

- Receive the file into `spool\incoming\{jobId}.xps`, verify size matches the header, then enqueue.
- Print with `System.Printing: new PrintServer().GetPrintQueue(name).AddJob(jobName, path, false)`. The *false* (fastCopy) lets Windows convert for non-XPS drivers.
- Accept both **.xps** and **.oxps** (the v4 writer on Win 8+ produces OpenXPS). Windows 8+ hosts print both; see pitfall 11.2 for a Windows 7 host.
- Reply with status JSON; poll `PrintSystemJobInfo` for up to 60 s to report printed/error.
- Process jobs one at a time per printer, in arrival order; use a simple in-memory queue.

Important: System.Printing is **not supported inside a Windows Service**. The Host role must run as a tray application in the logged-in user's session, auto-started at logon (registry Run key or a scheduled task "At log on"). Office PCs stay logged in during working hours, so this is acceptable. A true service using the native XpsPrint API is a Phase 2 option.

6. Protocol (version 1)

Channel	Port	Format
Discovery request	UDP broadcast 255.255.255.255:9150	ASCII text MSETU-DISCOVER 1; sent every 10 s while the client tab is open, plus on demand.
Discovery reply	UDP unicast back to sender	JSON: {"app": "MudranSetu", "v": 1, "host": "COUNTER1", "ip": "192.168.1.5", "port": 9151, "printers": [{"id": "...", "name": "HP LaserJet 1020", "friendly": "Counter 1 Laser", "status": "ready"}]}
Job transfer	TCP 9151, one job per connection	4 bytes ASCII MSTU, then 4-byte little-endian header length, then UTF-8 JSON header, then raw file bytes. Host answers one JSON line then closes.
Job header	-	{"type": "job", "v": 1, "printerId": "...", "jobId": "guid", "fileName": "job.xps", "format": "xps oxps", "size": 123456, "client": "COUNTER3", "user": "spm", "doc": "optional", "pin": "sha256-hex or empty"}
Job reply	-	{"ok": true, "jobId": "guid", "state": "queued printing printed error", "message": "human text"}
Status query	TCP 9151	Header {"type": "status", "jobId": "guid"}, no body; same reply format.

- Both ports are configurable in Settings but default as above. Never use 9100 (reserved for the Phase 2 raw relay) or 631.
- Optional shared **PIN** (Settings on both sides). Header carries SHA-256 of the PIN; host rejects mismatches with a clear message. Empty PIN = open on the LAN.
- Limits: max file 200 MB; connect timeout 5 s; transfer timeout 5 min; discovery timeout 2 s.
- Manual host entry (IP or name) must always be available because UDP broadcast does not cross subnets or VLANs.

7. User interface

- **Share my printers** tab: list of local printers with a checkbox and an editable friendly name; "Sharing is ON/OFF" toggle; today's job count.
- **Use shared printers** tab: discovered hosts and their printers; *Add to this PC / Remove* buttons; "Add by IP" box; last 10 jobs with status.
- **Settings** tab: PIN, ports, start with Windows, keep files for N hours, language (English now, Hindi later).
- **Diagnostics** tab: one button that copies OS version, .NET version, IPs, firewall rule state, printer/port list and the last 200 log lines to the clipboard, and a "Send test page" button per remote printer.
- Tray icon with balloon notifications for job sent / printed / failed. Closing the window minimises; Exit is in the tray menu.
- Keep every message in plain language. Error text must say what to do next ("Ask the Counter 1 PC to be switched on and try again").

8. Repository layout

MudranSetu/ README.md CLAUDE.md	short user guide with screenshots (later) condensed rules for Claude Code (Appendix A)
---------------------------------------	---

```

LICENSE                               MIT
MudranSetu.sln
src/MudranSetu.Core/   Protocol/, Discovery/, Spool/, Printing/, Config/, Logging/
src/MudranSetu.App/    Forms/, Tray/, Strings.resx, app.manifest (asInvoker)
src/MudranSetu.Elevate/ tiny console exe, requireAdministrator manifest;
                        only creates/removes printers & ports & firewall rules
tests/MudranSetu.Tests/
installer/MudranSetu.iss
docs/                  this PDF, protocol.md, testing.md
.github/workflows/build.yml windows-latest, setup-msbuild, nuget restore,
                        msbuild /p:Configuration=Release, upload artifact

```

Build locally with msbuild `MudranSetu.sln /p:Configuration=Release` (Visual Studio 2022 Build Tools with the .NET Framework 4.8 targeting pack). Commit after each milestone with a message like M2: client virtual printer and spool watcher.

9. Milestones and acceptance tests

#	Deliverable	Owner verifies by
M0	Solution scaffold, three projects build, CLAUDE.md, CI green, empty tray app runs.	Running the EXE on Win 10 shows a tray icon and an empty window.
M1	Host role: printer list, share toggle, TCP listener, AddJob printing. Plus <code>mstu-send.exe file.xps host-ip</code> test tool.	Printing an XPS file from another PC via the test tool produces paper (or a PDF via Microsoft Print to PDF on the host).
M2	Client role on Win 10/11: Elevate helper creates port + virtual printer; watcher ships jobs; status balloons.	Print from Notepad on the client; it comes out of the host printer within 10 s.
M3	UDP discovery, Add/Remove UI, manual IP entry, job history, config persistence.	A fresh client PC finds the host without typing anything.
M4	Windows 7 pass: driver name detection, 32-bit, FileSystemWatcher fallback, .oxps handling (see 11.2).	Win 7 client to Win 10 host and Win 10 client to Win 7 host both print.
M5	Inno Setup installer: .NET 4.8 check, firewall rules, autostart, clean uninstall that removes virtual printers.	Install and uninstall on a VM leaves no printers, ports, rules or folders behind.
M6	Hardening: PIN, retries, spool cleanup, Diagnostics tab, log rotation, plain-language errors.	Switch the host off mid-job; the client explains clearly and Retry works after the host returns.
P2	Phase 2: raw port-9100 relay, Hindi UI, optional service host, job counter per user.	After M6 is in daily use for two weeks.

10. Installer and deployment

- Inno Setup 6. One installer for both roles; a page asks "Will this PC share a printer?" to pre-tick the Host role.
- Check for .NET Framework 4.8 (registry `HKLM\SOFTWARE\Microsoft\NET Framework Setup\NDP\v4\Full Release >= 528040`). If missing, tell the user to run the bundled offline installer `ndp48-x86-x64-allos-enu.exe` (about 115 MB; keep it beside the setup EXE, not inside the repo).
- Windows 7 prerequisites before .NET 4.8 will install: SP1, KB4474419 and KB4490628 (SHA-2 signing support). The installer should detect Win 7 and show this list.
- Add firewall rules with `netsh advfirewall firewall add rule` for TCP 9151 and UDP 9150, profile Private+Domain, program-scoped.
- Autostart via `HKLM\...\Run` (all users) with the `/tray` switch.
- Uninstall runs `MudranSetu.Elevate.exe /remove-all` first, then deletes rules and folders. Never leave an orphan virtual printer.
- The EXE will be unsigned at first. Windows SmartScreen will warn; document the "More info, Run anyway" click in README. Code signing is a later decision.

11. Known pitfalls (read before coding)

11.1 Service vs tray

System.Printing and most spooler UI APIs fail or hang in Session 0. The host must be a tray app. Do not "fix" this by making a service.

11.2 .xps vs .oxps

Windows 7 ships the v3 XPS writer (.xps). Windows 8+ ship the v4 writer, which writes OpenXPS (.oxps). A Windows 7 **host** cannot print .oxps natively. Options, in order of preference:

- Recommend hosts on Windows 10/11 (they print both). Show a warning in the UI when a Win 7 host receives .oxps.
- On Win 8+ clients, try creating the virtual printer with the older "Microsoft XPS Document Writer" v3 driver if it is present in the driver store.
- Last resort: on Win 8+ clients use "Microsoft Print to PDF" and add a PDF renderer on the host (PdfiumViewer, net48). Ask the owner before adding this dependency.

11.3 Local Port file is overwritten per job

Two quick prints in a row can collide. Renaming on close (5.2) handles the common case. Also set the virtual printer to "Start printing after last page is spooled".

11.4 Elevation

AddPrinter, XcvData AddPort and netsh need admin. Keep all of that in **MudranSetu.Elevate.exe** with a *requireAdministrator* manifest, launched via ShellExecute with runas. The main app stays *asInvoker*. Pass instructions as command-line arguments, return results via exit code and a JSON file in the spool folder.

11.5 Networking

- Windows Firewall blocks both ports until rules exist; Diagnostics must show whether the rules are present.
- Broadcast discovery is subnet-only. Manual IP entry is mandatory, and the host IP may change if DHCP is used; store the host *name* too and re-resolve.
- Wi-Fi printers already on the LAN can still be shared through a host PC; the host simply has them installed as normal printers.

11.6 Other

- Copies count and duplex chosen in the client print dialog are lost by the XPS writer; document as a limitation, print one copy, and let the client UI offer "Copies" per job later.
- Paper size travels inside the XPS. If the host printer lacks Legal/A4 as configured, the driver may scale or reject; log the driver message verbatim.
- Old antivirus on Win 7 may quarantine an unsigned network-listening EXE; note in README how to whitelist the ProgramData folder and EXE.
- Never call `PrintServer` on the UI thread; it can block for seconds on a slow spooler.
- Log every step of a job with its *jobId* on both sides so a failed job can be traced from two log files.

12. Coding conventions and interaction rules

- Small classes, one responsibility each. Async/await for all network and file I/O; never block the UI thread.
- Every catch block logs the exception with *jobId* and context, then surfaces a plain-language message. No silent catches.
- Own minimal logger in Core (daily rolling file, 14-day retention). No logging frameworks.

- Unit-test the protocol framing, discovery JSON, config load/save and file-stability check. Print and spooler code is integration-tested by the owner.
- Before doing any of the following, stop and ask the owner: adding a NuGet package, changing target framework, changing ports, requiring internet, changing the data folder, or introducing a Windows Service.
- When the owner pastes an error or log, first restate what you think happened in one sentence, then propose the smallest fix.
- Write the README as you go: install steps, screenshots placeholders, troubleshooting table.

13. Glossary

Term	Meaning
Host	The PC that has the printer and runs MudranSetu in Share mode.
Client	A PC that prints to a remote printer through MudranSetu.
Virtual printer	A printer entry on the client that uses the built-in XPS writer driver and writes a file instead of printing.
Local Port	A Windows printer port type whose name is a file path; each job overwrites that file.
XPS / OpenXPS	Microsoft's page description formats; .xps from Windows 7's writer, .oxps from Windows 8+.
Spooler	The Windows print service that queues jobs and talks to drivers.
SO / PC names	Office PCs are usually named by role, e.g. COUNTER1, SPM, MAIL. Show PC names, not IPs, wherever possible.

Appendix A: starter CLAUDE.md

MudranSetu - CLAUDE.md

What this is

Windows tray app that shares printers across a LAN of mixed Windows 7/8/10/11 PCs without drivers or credentials on the client. Full brief: docs/MudranSetu_Claude_Code_Guide.pdf.

Hard rules

- C# on .NET Framework 4.8 only. WinForms. AnyCPU, Prefer 32-bit OFF.
- No NuGet except Newtonsoft.Json. Ask before adding anything.
- LAN only. No internet calls, no telemetry, no auto-update.
- Host role runs as a tray app in the user session, never as a Windows Service.
- Ports: UDP 9150 discovery, TCP 9151 jobs. Configurable, but never 9100 or 631.
- All admin work lives in src/MudranSetu.Elevate (requireAdministrator). Main app is asInvoker.
- Data in %ProgramData%\MudranSetu\{config.json, spool\, logs\}.
- All UI strings in Strings.resx.

How it works

Client: virtual printer (Microsoft XPS Document Writer driver, Local Port -> file) -> FileSystemWatcher -> TCP send. Host: receive -> PrintQueue.AddJob(path, fastCopy:false).

Workflow

- One milestone at a time (M0..M6 in the PDF). Build with:
msbuild MudranSetu.sln /p:Configuration=Release
- After each milestone: list exact manual test steps for the owner, who tests on Win 7 / Win 10 / Win 11 machines and pastes logs back.
- Every job step logs with jobId on both sides.
- Ask before: new dependency, framework change, port change, anything needing admin outside the Elevate helper, anything needing internet.

Style

Small classes, async I/O, no blocking UI thread, no silent catches, plain-language error messages that say what to do next.